

TEDEX V1.3.0: A Reproducible Workspace for Thermoelectric Data Analysis

Heyang Chen

Nanyang Techological University, Northwestern Thermoelectrics
te_agent_workspace

Abstract

TEDEX is a Python-based workspace for thermoelectric materials research. It organizes raw ZEM/LFA/XRD measurements, sample metadata, processed transport tables, publication-style figures, batch assessments, and local optimization reports in a single reproducible project layout. This document gives a compact system overview and a practical operating guide. It is intentionally formatted like a short CS/AI systems paper: the first half describes the architecture and dataflow, while the second half provides command-driven operation cards with fillable slots for bash commands, terminal output, and generated figures.

Keywords: thermoelectric materials; data pipeline; XRD; transport properties; reproducible analysis; Bayesian optimization; scientific plotting

1 Introduction

Modern thermoelectric experiments produce heterogeneous data: instrument exports from ZEM and LFA, XRD scans, sample ledgers, density and heat-capacity metadata, plotting recipes, extracted literature tables, and downstream reports. TEDEX provides a lightweight but structured workspace for turning those artifacts into traceable analysis products.

The main design principle is simple: runnable entry points stay at the project root, reusable implementation lives under `src/`, source data lives under `data/`, and generated artifacts are separated into `results/` and `outputs/`. This keeps interactive lab work fast while preserving enough structure for batch reprocessing and paper-figure generation.

2 System Overview

Figure 1 summarizes the intended workflow. Raw experimental files are synchronized into metadata ledgers, processed into sample-level CSV tables, visualized by dedicated plotting tools, and optionally passed to assessment or prediction modules.

3 Core Capabilities

Transport analysis. The raw-data workflow reads paired ZEM/LFA files, calculates transport properties, extracts compact sample features, and writes processed

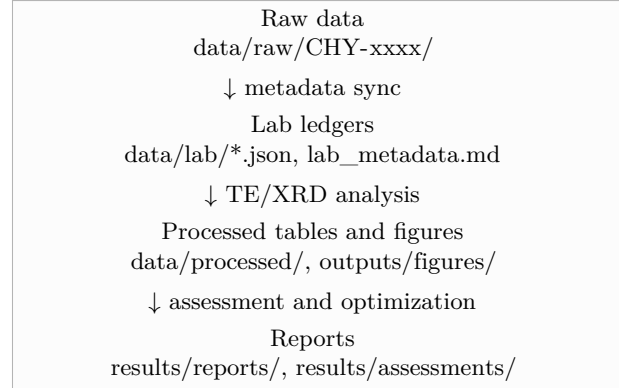


Figure 1: High-level TEDEX dataflow. Replace this schematic with a polished pipeline diagram if the document is used in a presentation or report.

CSV outputs. The main entry point is `run_analysis.py`.

TE plotting. `plot_te.py` renders single-property plots, combined summary figures, and inter-batch comparisons. It supports axis limits, selected property panels, multiple output formats, and no-preview batch rendering.

XRD plotting and lattice fitting. `plot_XRD.py` renders raw or normalized XRD scans, supports selected samples and optional PDF-card stick patterns, and writes figures under `outputs/figures/xrd/`. Lattice fitting utilities live under `src/tools/xrd_lattice.py`.

Flexible plotting. `flexible_plot.py` targets loosely structured CSV/TXT/XLSX files. It can render line, scatter, bar, grouped-bar, multi-panel, and dual-axis figures from direct CLI arguments or reusable JSON recipes.

Assessment and prediction. `assess_selected_batches.py` summarizes selected batches from extracted feature payloads. `bayesian_predict_te.py` fits local Gaussian-process models over lab or reference feature tables and ranks candidate compositions.

Agent-assisted interpretation. The `src/agents/` module adds model-assisted interpretation. It requires the Python dependencies in `requirements.txt` and a private local API key configured through `.env` or environment variables.

Table 1: Top-level project layout.

Path	Role
*.py	Root-level runnable entry scripts.
src/	Importable analysis, plotting, XRD, SPB, and agent modules.
scripts/	Secondary CLI utilities for synchronization and export.
data/raw/	Original instrument exports and unmodified source data.
data/processed/	Cleaned transport-property CSV files.
data/lab/	Batch ledgers, sample ledgers, and editable metadata.
configs/	Plot recipes and reusable runtime configurations.
results/	Reports, assessments, prediction tables, and payloads.
outputs/	Figures, tables, logs, slide assets, and one-off exports.

4 Environment Setup

All commands should be run from the workspace root:

Listing 1: Workspace setup.

```
cd ~/Users/chenheyang/Library/CloudStorage/OneDrive-
NorthwesternUniversity/02-Northwestern Lab/
te_agent_workspace"
pip install -r requirements.txt
```

Use the following commands to inspect the active interfaces:

Listing 2: Entrypoint help commands.

```
python run_analysis.py --help
python plot_te.py --help
python plot_XRD.py --help
python flexible_plot.py --help
python assess_selected_batches.py --help
python bayesian_predict_te.py --help
```

5 Operating Cards

Each operating card below is designed to be filled in after an actual run. The Command block records the exact bash command, the Terminal output block records the key log lines, and the Figure slot shows where to place the generated image.

5.1 Metadata Synchronization

Purpose: scan raw folders, update data/lab/batches.json and data/lab/samples.json, and optionally round-trip the editable Markdown ledger.

Listing 3: Command slot: metadata sync.

```
# TODO: replace with the exact command used in the run.
python scripts/sync_lab_metadata.py --dry-run
python scripts/sync_lab_metadata.py
python scripts/sync_lab_markdown.py export
# Edit data/lab/lab_metadata.md, then import edits:
python scripts/sync_lab_markdown.py
```

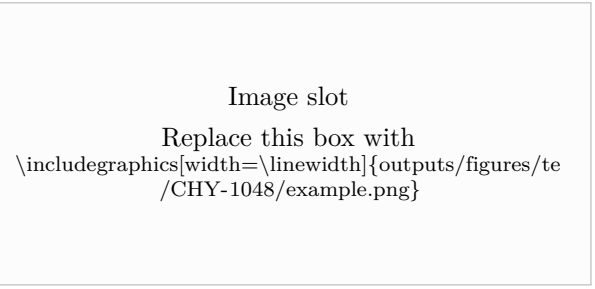


Figure 2: Figure slot for a representative TE output generated after raw-data analysis.

Listing 4: Terminal-output slot: metadata sync.

```
# TODO: paste the key output lines here.
# Example:
# Found batch CHY-1048
# Updated data/lab/batches.json
# Updated data/lab/samples.json
```

Notes slot. Record any manual metadata edits here, such as density, cp_value, composition labels, annealing conditions, or known sample issues.

5.2 Raw TE Analysis

Purpose: process ZEM/LFA raw data for one or more batches and write cleaned transport-property tables.

Listing 5: Command slot: TE raw-data analysis.

```
# Single batch
python run_analysis.py CHY-1048

# Multiple batches
python run_analysis.py CHY-1038 CHY-1040

# Selected samples within one batch
python run_analysis.py CHY-1054 --sample B C
```

Listing 6: Terminal-output slot: TE analysis.

```
# TODO: paste representative log lines here.
# Example:
# step1. Data processing
# Saved processed data to: data/processed/CHY-1048/...
# step2. SPB-derived transport columns
# Saved extracted features to: results/...
```

5.3 TE Plotting

Purpose: generate single-property and combined thermoelectric figures.

Listing 7: Command slot: TE plotting.

```
# Default single-property output
python plot_te.py CHY-1040 --no-show

# Combined summary figure
python plot_te.py CHY-1040 --plot-mode combined --no-show

# Inter-batch comparison with fixed axes
python plot_te.py CHY-1038 CHY-1040 \
--plot-mode combined \
--comparison-id CHY1038_vs_CHY1040 \
--xlim 300 800 \
--no-show
```

Listing 8: Terminal-output slot: TE plotting.

```
# TODO: paste saved-file lines here.
# Example:
# Saved: outputs/figures/te/CHY-1040/seebeck.svg
```

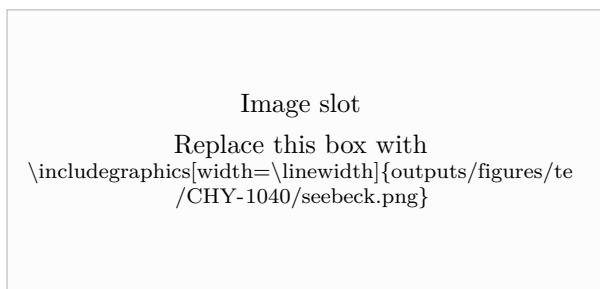


Figure 3: Figure slot for a TE property plot, such as Seebeck, conductivity, power factor, or zT .

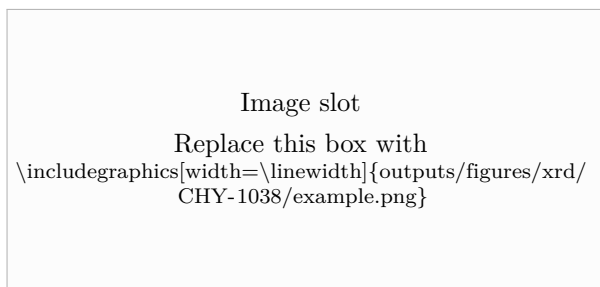


Figure 4: Figure slot for a stacked or normalized XRD plot.

```
# Saved: outputs/figures/te/inter-batch/CHY1038_vs_CHY1040
/...
```

5.4 XRD Plotting

Purpose: render XRD scans for a batch, selected samples, or a direct XRD folder; optionally overlay a cleaned PDF-card standard.

Listing 9: Command slot: XRD plotting.

```
# Whole batch
python plot_XRD.py CHY-1038 --no-show

# One normalized sample
python plot_XRD.py CHY-1038-A --mode normalized --no-show

# Selected suffixes from an XRD folder
python plot_XRD.py data/raw/CHY-1038/XRD \
--select A C \
--no-show

# With a PDF-card standard
python plot_XRD.py CHY-1038 \
--pdf-card CuInTe2_PDF_97_023_8958 \
--no-show
```

Listing 10: Terminal-output slot: XRD plotting.

```
# TODO: paste saved-file lines or selected sample list here.
# Example:
# Loaded 3 XRD files
# Saved: outputs/figures/xrd/CHY-1038/...
```

5.5 Flexible Plotting

Purpose: render figures from ad hoc CSV/TXT/XLSX inputs or reusable JSON recipes.

Listing 11: Command slot: flexible plotting.

```
# Direct CLI mode
python flexible_plot.py \
--data data/demo/flexible_plotting/
literature_comparison_loose.csv \
```

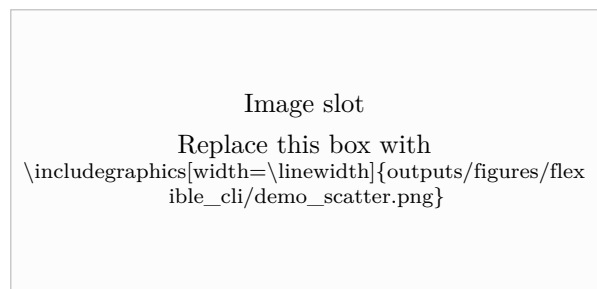


Figure 5: Figure slot for a flexible-plotting output.

```
--kind scatter \
--x "Ag fraction x" \
--y "Best zT" \
--xlabel "Ag fraction, $x$" \
--ylabel "$ZT_{\mathrm{max}}$" \
--stem demo_scatter \
--formats png pdf \
--no-show

# Recipe mode
python flexible_plot.py \
--recipe configs/flexible_plot_demos/temperature_seebeck_line.
json \
--no-show
```

Listing 12: Terminal-output slot: flexible plotting.

```
# TODO: paste generated files here.
# Example:
# outputs/figures/flexible_cli/demo_scatter.png
# outputs/figures/flexible_cli/demo_scatter_normalized.csv
```

5.6 Assessment and Bayesian Prediction

Purpose: summarize extracted transport features and rank candidate compositions for follow-up measurements.

Listing 13: Command slot: assessment and prediction.

```
# Assess selected batches
python assess_selected_batches.py CHY-1040 CHY-1048

# Assess every batch with extracted features
python assess_selected_batches.py --all

# Local GP prediction on lab data
python bayesian_predict_te.py --source lab --target ZT_max --
top 10

# Reference-data prediction
python bayesian_predict_te.py \
--source reference \
--reference-id GeSe \
--target ZT_max \
--pdf
```

Listing 14: Terminal-output slot: prediction.

```
# TODO: paste top candidates or report paths here.
# Example:
# Top candidate: ...
# Report: results/bayesian_predictions/...
# Figure: outputs/figures/bayesian_predictions/...
```

6 Recommended End-to-End Protocol

For a new experimental batch, use the following protocol:

1. Place raw files under data/raw/<batch_id>/. Keep original instrument exports unchanged.

2. Run metadata synchronization and fill missing density, heat capacity, composition, and synthesis annotations.
3. Process the raw TE data with `run_analysis.py`.
4. Render TE figures and inspect axis limits.
5. Render XRD figures and, when appropriate, compare with a PDF-card standard.
6. Run batch assessment and record the main finding in `results/`.
7. If enough structured data exists, run a Bayesian prediction pass and use the ranked candidates to plan follow-up samples.

Listing 15: End-to-end command template.

```
python scripts/sync_lab_metadata.py
python scripts/sync_lab_markdown.py export
# TODO: edit data/lab/lab_metadata.md
python scripts/sync_lab_markdown.py
python run_analysis.py CHY-1048
python plot_te.py CHY-1048 --seebeck --no-show
python plot_XRD.py CHY-1048 --no-show
python assess_selected_batches.py CHY-1048
```

7 Reproducibility Notes

Generated outputs should be stored in predictable locations. Standard TE plots belong under `outputs/figure_s/te/`; XRD plots belong under `outputs/figures/xrd/`; flexible one-off plots belong under `outputs/figures/flexible_cli/` or `outputs/figures/flexible/`; assessment reports belong under `results/assessments/`; Bayesian reports belong under `results/bayesian_predictions/`. Private credentials and local-only configuration files should remain outside versioned or public artifacts.

8 Editable Example Slots

Use the following minimal pattern when adding a new operation example. Copy the three blocks together so each example records its command, textual output, and visual output.

Listing 16: Generic command slot.

```
# TODO: paste the exact bash command here.
python your_script.py --your-option value --no-show
```

Listing 17: Generic terminal-output slot.

```
# TODO: paste only the important output lines here.
# Keep this short enough to fit in one column.
```

9 Pointers

More detailed command references are maintained in `docs/COMMANDS.md`, `docs/FLEXIBLE_PLOTTING.md`, and `docs/PROJECT_STRUCTURE.md`. When the documentation and script behavior disagree, prefer the current `--help` output from the corresponding root entry point.

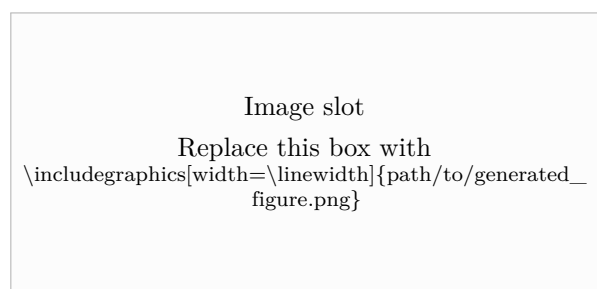


Figure 6: Generic output-image slot. Replace the placeholder with an actual figure once the command has been run.